TÀI LIỆU HỌC VÀ CHUẨN BỊ THUYẾT TRÌNH

Đào Văn Tâm (B25CHHT112) · Nhóm 8 · Tìm kiếm và Truy xuất thông tin · Đề tài LinCIR

1. Vai trò của bạn

Trình bày toàn bộ 16 slide (khoảng 15 phút) và điều phối hỏi đáp. Phần việc riêng: chuyển đổi mã nguồn LinCIR sang môi trường mới, kiểm chứng tương đương với mô hình gốc; vận hành thực nghiệm trên Kaggle: lập lịch chạy song song trong hạn mức GPU, khắc phục nghẽn nạp dữ liệu.

Slide	Nội dung	Thời gian
1	Bìa, giới thiệu và tóm tắt dễ hiểu	1:10
2	Phân công công việc	0:30
3	Bài toán CIR, câu hỏi của nhóm	1:00
4	Độ đo R@K, mAP@K (liên hệ chương I)	1:00
5	Ý tưởng token giả	0:50
6	Tóm tắt bài báo LinCIR	1:00
7	Self-Masking Projection	1:20
8	Nhiễu và modality gap	1:00
9	Cải tiến A: InfoNCE (H1)	1:00
10	Cải tiến B: nhiễu có hướng (H2, H3)	1:00
11	Dựng mã nguồn, chạy trên Kaggle	1:00
12	Tái hiện, khác biệt bài báo – mã nguồn	0:50
13	Kết quả chính, kiểm định	1:30
14	Vì sao: độ nhất quán của φ	1:20
15	Kết luận H1–H3, ví dụ truy xuất	1:20
16	Hỏi đáp	—

2. Bức tranh chung của đề tài (cả nhóm cần nắm)

Tóm tắt dễ hiểu (đọc đầu tiên)

Bài báo nói về điều gì

Bài toán. Bạn đang xem ảnh một chiếc váy hoa màu xanh trên trang bán hàng. Bạn muốn tìm một chiếc gần giống nhưng có thêm vài thay đổi, và gõ: "màu bạc, bóng hơn, tay ngắn hơn". Hệ thống phải tìm trong hàng chục nghìn ảnh ra chiếc váy vừa giữ kiểu dáng của ảnh ban đầu, vừa có đúng các thay đổi đó. Bài toán tìm kiếm bằng một ảnh kèm một câu mô tả thay đổi như vậy gọi là **truy xuất ảnh kết hợp**.

Bài toán này khó vì câu "màu bạc, tay ngắn hơn" không tự mô tả được chiếc váy cần tìm. Nó chỉ có nghĩa khi đặt cạnh ảnh ban đầu. Hệ thống phải hiểu cả ảnh lẫn câu cùng lúc.

Cách làm cũ và chỗ tốn kém. Cách đơn giản là thuê người tạo dữ liệu mẫu: mỗi mẫu gồm ảnh ban đầu, câu mô tả thay đổi và ảnh đúng cần tìm. Việc này rất tốn công, nên bộ dữ liệu

lớn nhất cũng chỉ có khoảng 46 nghìn mẫu. Mô hình học từ dữ liệu thời trang thì cũng chỉ dùng tốt cho thời trang.

Một số phương pháp gần đây tránh được việc thuê người gán nhãn. Ý tưởng chung là đổi ảnh ban đầu thành một "từ đặc biệt", rồi ghép từ đó vào câu tìm kiếm, ví dụ: "ảnh của [từ đặc biệt], màu bạc, tay ngắn hơn". Một mô hình có sẵn tên CLIP, vốn đã được huấn luyện trên hàng trăm triệu cặp ảnh và câu, sẽ biến câu này thành con số rồi so với từng ảnh trong kho. Chỉ cần dạy một mạng nhỏ (gọi là φ) cách đổi ảnh thành từ đặc biệt.

Tuy vậy, các phương pháp trước vẫn dùng ảnh khi dạy mạng φ. Việc này nặng: bộ ảnh huấn luyện chiếm khoảng 430 GB, trong khi phần chữ đi kèm chỉ 125 MB. Chúng cũng chỉ học với một mẫu câu cố định.

Ý tưởng của LinCIR. Tác giả (công ty NAVER, Hàn Quốc, công bố năm 2024) đề xuất dạy mạng φ **chỉ bằng chữ, không cần ảnh**. Cách làm:

1. Lấy một câu, ví dụ "con mèo xám ngủ trên gối".
2. Che các từ quan trọng, câu thành "[…] ngủ trên […]".
3. Cho mạng φ đọc câu gốc rồi tạo ra một từ đặc biệt điền vào chỗ trống.
4. Yêu cầu câu đã điền phải mang nghĩa giống câu gốc.

Muốn làm được, mạng φ phải học cách gói nội dung chính của cả câu vào một từ. Khi dùng thật, đưa ảnh vào thay cho câu, mạng φ tạo ra từ đặc biệt đại diện cho ảnh.

Có một trở ngại: trong CLIP, con số biểu diễn ảnh và con số biểu diễn chữ nằm ở hai vùng khác nhau, dù cùng nói về một thứ. Mạng φ học trên chữ nhưng lại phải xử lý ảnh. Tác giả giải quyết bằng cách thêm nhiễu ngẫu nhiên vào đầu vào khi huấn luyện, để mạng quen với những đầu vào lệch khỏi vùng chữ.

Kết quả bài báo. LinCIR đạt kết quả tốt nhất trên bốn bộ đánh giá, huấn luyện nhanh hơn các phương pháp trước 6 đến 8 lần, và trên bộ thời trang còn vượt một phương pháp phải dùng dữ liệu do người gán nhãn.

Nhóm 8 đã làm gì

1. Dựng lại và kiểm tra. Mã nguồn của tác giả viết cho phiên bản thư viện cũ, không chạy được trên máy hiện nay. Nhóm viết lại phần bị hỏng, sửa 4 lỗi, rồi kiểm tra: phần viết lại cho ra con số giống hệt bản gốc. Sau đó nhóm chạy lại mô hình tác giả công bố bằng mã của nhóm. Kết quả khớp bài báo, lệch không quá 0,77 điểm. Như vậy cách đánh giá của nhóm đáng tin.

2. Đặt câu hỏi. LinCIR có hai thành phần chính: cách chấm điểm khi dạy mạng φ, và phần nhiễu. Nhóm hỏi: sửa một trong hai thành phần này thì kết quả có tốt hơn không?

3. Hai cải tiến.

- **Cải tiến A (sửa cách chấm điểm).** Cách chấm gốc chỉ yêu cầu câu đã điền giống câu gốc. Nó không phạt trường hợp câu đã điền cũng giống nhiều câu khác. Nhóm thêm một yêu cầu: câu đã điền phải giống câu gốc **hơn** 511 câu khác đang xét cùng lúc. Kỳ vọng là mạng φ phân biệt rõ hơn, nên tìm kiếm chính xác hơn.

- **Cải tiến B (sửa phần nhiễu).** Nhiễu gốc lệch ngẫu nhiên theo mọi hướng. Nhóm đo hướng từ vùng chữ sang vùng ảnh, rồi cho nhiễu lệch có chủ đích về phía vùng ảnh. Khi đó mạng φ được luyện với đầu vào gần với ảnh thật mà nó sẽ gặp khi dùng.
- **Kết hợp A+B** để xem hai cải tiến có cộng thêm cho nhau không.

4. Thử nghiệm công bằng. Nhóm chạy cả bốn phiên bản (bản gốc, A, B, A+B) với cùng điều kiện, mỗi phiên bản 3 lần với khởi tạo ngẫu nhiên khác nhau, tổng cộng 12 lần. Thực nghiệm chạy trên máy chủ GPU miễn phí của Kaggle, dùng gần 23 trên 30 giờ được phép mỗi tuần. Kết quả được đánh giá trên hai bộ dữ liệu: thời trang (FashionIQ) và ảnh đời thực (CIRCO). Mọi chênh lệch đều được kiểm định thống kê để chắc rằng không do ngẫu nhiên.

Kết quả

- **Cải tiến B tốt hơn bản gốc, và có ý nghĩa thống kê.** Trên bộ thời trang, cứ 1.000 lượt tìm kiếm thì B tìm đúng thêm khoảng 6 lượt trong 10 kết quả đầu. B chỉ huấn luyện bằng một phần tư thời gian nhưng vẫn vượt cả mô hình tác giả công bố.
- **Cải tiến A làm kết quả kém đi,** rõ nhất trên ảnh đời thực (điểm giảm từ 10,47 xuống 8,02). A+B cũng kém.

Vì sao. Nhóm đưa cùng một cặp ảnh và câu mô tả vào mạng φ, rồi đo hai đầu ra giống nhau đến mức nào:

- Với B, hai đầu ra giống nhau nhất: mạng xử lý ảnh nhất quán với cách nó xử lý chữ.
- Với A, độ giống giảm mạnh. Yêu cầu "phân biệt với 511 câu khác" khiến mạng học những chi tiết chỉ có ở chữ, mà ảnh không có.

Kết luận của nhóm. Với LinCIR, điều quan trọng nhất là mạng φ xử lý ảnh nhất quán với chữ. Làm cho mạng phân biệt các câu tốt hơn thì không giúp ích. Kết luận này có số đo trực tiếp và kiểm định thống kê làm căn cứ, không phải phỏng đoán.

Tóm tắt kỹ thuật

Bài toán. Truy xuất ảnh kết hợp (Composed Image Retrieval – CIR): truy vấn gồm một ảnh tham chiếu và một câu mô tả thay đổi (“chiếc váy này nhưng màu bạc, tay ngắn hơn”); hệ thống trả về ảnh thỏa cả hai. Dữ liệu bộ ba ⟨ảnh, câu, ảnh đích⟩ đắt và nhỏ, nên hướng **zero-shot** huấn luyện không cần bộ ba.

Bài báo. LinCIR (Gu và cộng sự, CVPR 2024) học một mạng chiếu φ (MLP 3 lớp) biến một vector thành “từ giả” [$] trong không gian token của CLIP, **chỉ bằng văn bản**: che các từ khóa của câu bằng [$] = φ(vector của câu + nhiễu) và ép câu đã che có cùng vector với câu gốc (Self-Masking Projection, mất mát MSE). Khi truy vấn, thay vector câu bằng vector ảnh: “a photo of [$] that {câu điều chỉnh}” → tìm ảnh theo cosine.

Nhóm đã làm. (1) Chuyển mã nguồn chính thức sang Python 3.13 / transformers 5.x, kiểm chứng sai lệch 0 so với mô hình gốc. (2) Tái hiện: checkpoint chính thức đạt FashionIQ R@10 = 26,35 / R@50 = 46,12 (bài báo 26,28 / 46,49). (3) Hai cải tiến: **A** – thêm mất mát đối sánh InfoNCE; **B** – nhiễu có hướng theo modality gap. (4) Huấn luyện baseline, A, B, A+B

trên Kaggle (2 × T4), mỗi biến thể 3 seed, 5.000 bước; đánh giá FashionIQ val + CIRCO val; kiểm định thống kê theo truy vấn.

Kết quả chính. B (nhiễu có hướng) tốt hơn baseline có ý nghĩa thống kê: FashionIQ R@10 +0,60 ($p = 0,006$), R@50 +1,27 ($p < 0,001$), CIRCO mAP@25/50 tăng có ý nghĩa; B còn vượt checkpoint chính thức 20.000 bước. **A (InfoNCE) không tốt hơn** trên FashionIQ và **kém rõ trên CIRCO** (mAP@5 −2,45, KTC 95% nằm dưới 0); A+B tương tự A. Giải thích: độ nhất quán của φ giữa ảnh và văn bản (cosine) B 0,84 > baseline 0,80 > A 0,64 — cùng thứ tự với CIRCO. Kết luận: H2 được ủng hộ, H1 và H3 bị bác bỏ.

Hạng mục	Số liệu / cấu hình
Backbone	CLIP ViT-L/14 (đóng băng); chỉ huấn luyện φ
Huấn luyện	AdamW, lr 1e-4, wd 0,01, batch 512, dropout 0,5, 5.000 bước (bài báo 20.000)
Dữ liệu train	Chỉ văn bản: chú thích CC3M + 2,47 triệu prompt Stable Diffusion + prompt Midjourney
Đánh giá	FashionIQ val (6.016 truy vấn, 3 nhóm, R@10/R@50); CIRCO val (220 truy vấn, 123.403 ảnh, mAP@K)
Tái hiện	Checkpoint chính thức: FIQ R@10 26,35 vs 26,28; R@50 46,12 vs 46,49 (lệch lớn nhất 0,77 điểm)
Cải tiến A	$L = MSE + 0,5 \cdot InfoNCE$ ($\tau = 0,07$, 511 mẫu âm trong batch)
Cải tiến B	$n = 0,5 \cdot \Delta + u \cdot \varepsilon$; Δ = tâm ảnh − tâm văn bản trên 5.000 cặp COCO ($\|\Delta\| = 13,87$)
Giao thức	3 seed (12345, 2024, 7), checkpoint bước cuối, paired bootstrap 10.000 lần + Wilcoxon, kết luận khi $p < 0,05$
Tài nguyên	Kaggle 2 × T4 16 GB, 30 giờ GPU/tuần, giới hạn 12 giờ/phiên; 1,45 s/bước, 2,0 giờ/run
Kết quả (TB 3 seed)	FIQ R@10: baseline 25,90 · A 25,54 · B 26,50 · A+B 25,44; CIRCO mAP@5: 10,47 · 8,02 · 10,94 · 7,81

3. Trình bày rõ ràng, dễ hiểu (đọc trước khi tập)

Mẹo thuyết trình

- **Mở đầu bằng mạch chính trong 3 câu:** (1) Truy xuất ảnh kết hợp cần dữ liệu có nhãn rất tốn kém. (2) LinCIR huấn luyện chỉ bằng văn bản nên không cần dữ liệu đó. (3) Nhóm tái hiện LinCIR, thử hai cải tiến, chứng minh bằng kiểm định rằng một cải tiến tốt hơn và giải thích vì sao cải tiến còn lại kém hơn.
- **Mỗi slide theo trình tự:** nêu ý chính (tiêu đề) → nêu một con số → giải thích con số đó có nghĩa gì → nối sang slide sau. Không đọc toàn bộ bảng; chỉ nêu dòng hoặc cột cần thiết.
- **Diễn giải con số ngay khi nêu:** R@10 = 26 nghĩa là trong 26% số truy vấn, ảnh đúng nằm trong 10 kết quả đầu tiên; tăng 0,60 điểm R@10 tương đương thêm khoảng 6 truy vấn tìm đúng trên mỗi 1.000 truy vấn.

- **Thuật ngữ:** nêu tên tiếng Việt kèm tên tiếng Anh và định nghĩa một câu ở lần xuất hiện đầu tiên; các lần sau dùng đúng một tên đó.
- **Khi nói, làm tròn hợp lý** ("khoảng 26%", "giảm khoảng 23%"); số chính xác đã có trên slide và trong báo cáo.
- **Ngắt nghỉ sau kết luận chính** (slide 13) trước khi chuyển sang phần giải thích nguyên nhân.
- **Trả lời câu hỏi:** câu đầu nêu thẳng kết luận, câu sau nêu căn cứ (số liệu, mục trong báo cáo).

Slide của bạn: ý chính và con số cần nêu

Slide	Ý chính nói thành lời	Một con số và ý nghĩa
1	Giới thiệu đề tài, rồi tóm tắt bằng ví dụ chiếc váy: bài toán tìm ảnh theo ảnh và câu sửa đổi; LinCIR dạy máy chỉ bằng văn bản, không cần ảnh và người gán nhãn; nhóm dựng lại mã, thử hai cải tiến: B tốt hơn có ý nghĩa thống kê, A làm kết quả kém đi. Nêu lộ trình: bài toán, phương pháp, hai cải tiến, thực nghiệm, lý do của kết quả.	Huấn luyện nhanh hơn các cách trước 6 đến 8 lần; B tăng FashionIQ R@10 từ 25,90 lên 26,50
2	Phân công: Tâm chuyển đổi mã nguồn và chạy thực nghiệm trên Kaggle; Trung làm cải tiến A và đường học; Tùng tái hiện và đối chiếu bài báo – mã nguồn; Dũng làm giao thức đánh giá, kiểm định và cải tiến B; Nou làm tổng quan lý thuyết, phân tích định tính và thiết kế slides. Tâm trình bày, khi hỏi đáp thành viên phụ trách trả lời phần của mình.	—
3	Truy vấn gồm một ảnh tham chiếu và một câu chỉ mô tả phần thay đổi; hệ thống phải tìm ảnh giữ đặc điểm của ảnh tham chiếu và áp dụng thay đổi. Bài báo LinCIR giải bài toán này mà không cần bộ ba gán nhãn. Câu hỏi của nhóm: sửa hàm mất mát (A) hoặc sửa nhiễu (B) có giúp LinCIR truy xuất tốt hơn không.	FashionIQ chỉ có 46,6 nghìn bộ ba gán nhãn, CIRR 28,8 nghìn
4	CIR vẫn là truy xuất có xếp hạng như chương I. R@K là tỉ lệ truy vấn có ảnh đích trong K kết quả đầu, dùng cho FashionIQ (1 ảnh đúng). AP@K lấy Precision tại các vị trí có ảnh đúng, chia cho min(K, số ảnh đúng); mAP@K là trung bình trên các truy vấn, dùng cho CIRCO (nhiều ảnh đúng). Khác chương I: so khớp vector bằng cosine thay vì khớp từ. Mọi so sánh được kiểm định bootstrap và Wilcoxon theo từng truy vấn.	R@10 = 26 nghĩa là 26% số truy vấn có ảnh đích trong 10 kết quả đầu
5	Ảnh tham chiếu qua bộ mã hóa ảnh rồi qua mạng chiếu φ thành một token giả [$], ghép vào câu "a photo of [$] that …"; bộ mã hóa văn bản tạo vector truy vấn để xếp hạng ảnh theo cosine. Chỉ cần học φ, CLIP giữ nguyên.	—
6	Pic2Word và SEARLE học φ nhưng cần ảnh khi huấn luyện và chỉ học một câu mẫu cố định. LinCIR học φ chỉ từ văn bản bằng Self-Masking Projection và thêm nhiễu; bài báo đạt kết quả tốt nhất trên 4 bộ đánh giá, huấn luyện nhanh hơn và vượt cả phương pháp có	Nhanh hơn Pic2Word 6 lần, SEARLE 8,4 lần với ViT-L

	giám sát trên FashionIQ.	
7	Che tính từ, danh từ trong câu bằng [$], thay [$] bằng token do φ tạo từ chính câu gốc, rồi yêu cầu câu đã che có cùng vector với câu gốc (MSE). Ví dụ "gray cat sleeps on a pillow" thành "[$] sleeps on [$]". Nhờ vậy [$] học được bản tóm tắt một từ của câu.	—
8	Vector ảnh và vector văn bản của CLIP nằm ở hai vùng tách biệt. φ học trên văn bản nhưng khi suy luận nhận vector ảnh, nên LinCIR thêm nhiễu u · ε khi huấn luyện để φ chịu được độ lệch này.	Tâm hai vùng cách nhau 0,82; ảnh và chú thích của chính nó chỉ có cosine 0,26
9	Cải tiến A sửa hàm mất mát: MSE chỉ kéo câu đã che về câu gốc, không phạt khi nó cũng gần các câu khác, trong khi đánh giá là xếp hạng. Thêm InfoNCE với 511 câu khác trong batch làm mẫu âm. Giả thuyết H1: A tăng R@K, mAP@K so với baseline.	Chi phí chỉ tăng 2% thời gian mỗi bước (1,474 so với 1,448 giây)
10	Cải tiến B sửa nhiễu: nhiễu của LinCIR đều theo mọi hướng, không dịch về phía vùng ảnh. B cộng thêm 0,5 · Δ, với Δ là hiệu giữa tâm vector ảnh và tâm vector văn bản, tính trên COCO val2017 khác tập đánh giá. Giả thuyết H2: φ nhất quán hơn giữa ảnh và văn bản nên R@K, mAP@K tăng. H3: A và B sửa hai thành phần khác nhau nên A+B có thể cộng hưởng.	Độ lớn của Δ là 13,87, tính trên 5.000 cặp ảnh – chú thích
11	Mã gốc không chạy với thư viện hiện tại; nhóm viết lại encoder văn bản, thay tokenizer, sửa 4 lỗi và kiểm chứng sai lệch bằng 0. Thực nghiệm: 4 biến thể × 3 seed, 5.000 bước, chỉ khác thành phần cải tiến, chạy trên 2 GPU T4 của Kaggle.	Sai lệch 0,00 so với CLIP gốc; nạp dữ liệu nhanh gấp 3,1 lần
12	Trước khi cải tiến, nhóm đánh giá lại checkpoint chính thức bằng mã đã chuyển đổi; kết quả khớp bài báo. Ba điểm bài báo khác mã nguồn; nhóm theo mã vì mã tạo ra số liệu công bố.	Lệch lớn nhất 0,77 điểm phần trăm
13	So với baseline nhóm tự huấn luyện. B cao nhất ở mọi cột và tốt hơn có ý nghĩa thống kê; B 5.000 bước vượt cả checkpoint chính thức 20.000 bước. A và A+B kém hơn trên CIRCO.	B tăng 0,60 điểm R@10 (thêm khoảng 6 truy vấn đúng trên 1.000), p = 0,006
14	Đo độ tương đồng giữa đầu ra của φ khi nhận ảnh và khi nhận văn bản của cùng một cặp; thứ tự trùng với thứ tự CIRCO mAP@5. InfoNCE buộc [$] mang đặc trưng phân biệt các câu, không chuyển được sang ảnh. A thấp hơn baseline ở mọi mốc huấn luyện.	B 0,84, baseline 0,80, A 0,64
15	Ví dụ: B xếp đúng ảnh đích ở hạng 1, baseline thì không. Kết luận: H2 được ủng hộ, H1 và H3 bị bác bỏ; chất lượng phụ thuộc chủ yếu vào việc φ xử lý ảnh nhất quán với văn bản. Hạn chế: 5.000 bước, chưa quét λ, α, chưa có CIRR.	CIRCO mAP@5: B 10,94, A 8,02

Thuật ngữ: định nghĩa ngắn

Thuật ngữ	Định nghĩa	Ví dụ trong đề tài

Truy xuất ảnh kết hợp (CIR)	Tìm ảnh bằng truy vấn gồm một ảnh tham chiếu và một câu mô tả thay đổi	Ảnh váy hoa xanh + "màu bạc, tay ngắn hơn" → tìm ảnh váy bạc tay ngắn
Zero-shot	Mô hình không được huấn luyện trên dữ liệu có nhãn của bài toán đích, được đánh giá trực tiếp	LinCIR không dùng bộ ba nào của FashionIQ hay CIRCO khi huấn luyện
Bộ ba có nhãn	Ảnh tham chiếu – câu điều chỉnh – ảnh đích, do người gán thủ công	FashionIQ có khoảng 46,6 nghìn bộ ba huấn luyện
CLIP	Mô hình gồm bộ mã hóa ảnh và bộ mã hóa văn bản, đưa ảnh và câu mô tả tương ứng về gần nhau	Nhóm dùng CLIP ViT-L/14, giữ nguyên không huấn luyện lại
Vector nhúng	Dãy số (768 chiều với ViT-L) biểu diễn nội dung của một ảnh hoặc một câu	Mỗi ảnh trong kho được mã hóa thành một vector
Độ tương đồng cosine	Mức độ cùng hướng của hai vector, càng gần 1 càng giống	Kết quả truy xuất được xếp theo cosine giữa truy vấn và từng ảnh
Token giả [$]	Một vector đặt vào vị trí của một từ trong câu, đại diện cho ảnh tham chiếu	Câu truy vấn: "a photo of [$] that is silver with shorter sleeves"
Mạng chiếu φ	Mạng nơ-ron nhỏ (3 lớp) biến vector thành token giả; là thành phần duy nhất được huấn luyện	Cả 4 biến thể của nhóm chỉ khác nhau ở cách huấn luyện φ
Self-Masking Projection (SMP)	Che các cụm tính từ – danh từ của câu bằng [$] và huấn luyện φ để câu đã che giữ nguyên vector của câu gốc	"gray cat sleeps on a pillow" → "[$] sleeps on [$]"
Khoảng cách ảnh – văn bản (modality gap)	Vector ảnh và vector văn bản của CLIP nằm ở hai vùng tách biệt	Ảnh và chú thích của chính nó chỉ có cosine trung bình 0,26 (5.000 cặp COCO)
Nhiễu khi huấn luyện	Thành phần ngẫu nhiên cộng vào vector văn bản để φ chịu được khác biệt giữa ảnh và văn bản	LinCIR dùng $n = u \cdot \varepsilon$, $u \sim U(0,1)$, $\varepsilon \sim N(0, I)$
Cải tiến B – nhiễu có hướng	Cộng thêm vector Δ (tâm vector ảnh trừ tâm vector văn bản) vào nhiễu, đẩy đầu vào của φ về phía vùng ảnh	$n = 0{,}5 \cdot \Delta + u \cdot \varepsilon$, Δ tính trên 5.000 cặp COCO
Cải tiến A – InfoNCE	Hàm mất mát đối sánh: mỗi câu phải gần chính nó hơn 511 câu khác trong cùng batch	$L = MSE + 0{,}5 \cdot InfoNCE$
Baseline	LinCIR gốc do nhóm tự huấn luyện, cùng cấu hình với các biến thể, dùng làm mốc so sánh	5.000 bước, 3 seed
Checkpoint	Trọng số của mô hình được lưu tại một thời điểm huấn luyện	Checkpoint chính thức của tác giả huấn luyện 20.000 bước
Bước huấn luyện	Một lần cập nhật trọng số trên một batch 512 câu	5.000 bước mất khoảng 2 giờ trên GPU T4
Seed	Giá trị khởi tạo bộ sinh số ngẫu nhiên; khác seed thì thứ tự dữ liệu và	Mỗi biến thể chạy với 3 seed: 12345, 2024, 7

	nhiễu khác nhau	
Độ lệch chuẩn (±)	Mức dao động của kết quả giữa các lần chạy khác seed	B: R@10 = 26,50 ± 0,04 — rất ổn định
R@10, R@50 (Recall@K)	Tỷ lệ truy vấn có ảnh đích nằm trong K kết quả đầu	FashionIQ chỉ có một ảnh đúng cho mỗi truy vấn
mAP@5	Trung bình độ chính xác trên K = 5 kết quả đầu, dùng khi mỗi truy vấn có nhiều ảnh đúng	CIRCO có trung bình 4,16 ảnh đúng mỗi truy vấn
p-value	Xác suất quan sát được mức chênh lệch như vậy nếu thực tế hai phương pháp ngang nhau	B so với baseline, FashionIQ R@10: p = 0,006
Khoảng tin cậy 95% (KTC)	Khoảng giá trị của mức chênh lệch thật với độ tin cậy 95%; nếu toàn bộ khoảng lớn hơn 0 thì cải tiến tốt hơn	B, R@50: Δ = +1,27, KTC [+0,81; +1,73]
Paired bootstrap	Lấy mẫu lại các truy vấn 10.000 lần, tính lại mức chênh mỗi lần để ước lượng p-value và KTC	So từng truy vấn của B với cùng truy vấn đó của baseline
Âm tính giả	Ảnh thực tế thỏa truy vấn nhưng không được gán nhãn là đáp án	Lý do bài báo khuyến nghị dùng mAP và K lớn thay vì R@1
Ablation	So sánh các biến thể bật/tắt từng thành phần để tách đóng góp của mỗi thành phần	Baseline, A, B, A+B

Cách đọc ký hiệu trên slide

Lần đầu gặp [$] trên slide, nói thêm: "ký hiệu đô-la trong ngoặc vuông là token giả"; các lần sau chỉ nói "token giả". Ví dụ câu nói: "Mạng phi biến ảnh tham chiếu thành một token giả, ghép vào câu mô tả thay đổi."

Ký hiệu	**Đọc là**	**Nghĩa**
φ	"phi"	Mạng chiếu 3 lớp, phần duy nhất được huấn luyện; biến một vector (ảnh hoặc câu) thành một token
[$]	"token giả" (hoặc "token đô-la")	Vị trí trong câu được thay bằng đầu ra của φ; "a photo of [$] that is blue" đọc là "a photo of token giả that is blue"
Δ	"đen-ta"	Vector chênh lệch: tâm vector ảnh trừ tâm vector văn bản
ε	"ép-xi-lon"	Nhiễu ngẫu nhiên theo phân phối chuẩn
u · ε	"u nhân ép-xi-lon"	Nhiễu của LinCIR; u là hệ số ngẫu nhiên từ 0 đến 1
α	"an-pha"	Hệ số của Δ trong cải tiến B (0,5)
λ	"lam-đa"	Trọng số của InfoNCE trong cải tiến A (0,5)
τ	"tô"	Nhiệt độ của InfoNCE (0,07)
z_c	"z c"	Vector của câu gốc
ẑ_c	"z c mũ"	Vector của câu đã che từ khóa bằng token giả
L_SMP	"hàm mất mát SMP"	Hàm mất mát MSE của Self-Masking

		Projection
R@K, R@10	“Recall tại K”, “Recall tại 10”	Tỉ lệ truy vấn có ảnh đích trong K kết quả đầu
mAP@5	“m-A-P tại 5”	Độ chính xác trung bình trên 5 kết quả đầu
±	“cộng trừ”	Độ lệch chuẩn qua 3 seed; “26,50 ± 0,04” đọc là “26,50 cộng trừ 0,04”
p = 0,006	“p bằng không phẩy không không sáu”	Xác suất thấy mức chênh này nếu cải tiến thật ra không có tác dụng
InfoNCE, CLIP	“in-phô en-xi-i”, “clíp”	Đọc thẳng tên tiếng Anh; tương tự baseline, seed, batch

4. Kiến thức và nội dung công việc

Bộ mã hóa văn bản của CLIP

- Câu → tokenizer BPE → chuỗi mã token (tối đa 77, bắt đầu 49406, kết thúc EOS 49407) → lớp nhúng token + nhúng vị trí → 12 khối Transformer có **mặt nạ nhân quả** (mỗi token chỉ nhìn các token phía trước) → LayerNorm cuối → lấy vector tại vị trí EOS → phép chiếu tuyến tính → vector văn bản z_c (768 chiều với ViT-L).
- Vị trí EOS được tìm bằng argmax của mã token vì 49407 là mã lớn nhất trong câu. Do mặt nạ nhân quả, các token đệm phía sau EOS không ảnh hưởng tới kết quả.
- LinCIR chèn token giả bằng cách: ở mọi vị trí có mã 259 (ký tự “$”), thay vector nhúng token bằng $\hat{e} = \varphi(\cdot)$ trước khi đi qua các khối Transformer.

Vì sao phải viết lại

- Mã gốc gọi hàm nội bộ text_model.encoder(..., causal_attention_mask=...) của transformers 4.34; ở transformers 5.x tham số này không còn. Nhóm viết lại lượt truyền xuôi từ các mô-đun con có tên ổn định qua các phiên bản: layer_norm1, q/k/v/out_proj, layer_norm2, mlp, với scaled_dot_product_attention(is_causal=True).
- Gói clip của OpenAI (chỉ dùng clip.tokenize) khó cài trên Python ≥ 3.13 → thay bằng tokenizer CLIP của HuggingFace (cùng bảng từ vựng BPE, “$” vẫn là mã 259).

Kiểm chứng tương đương (cách chứng minh “đúng”)

- So vector của encoder viết lại với CLIPTextModelWithProjection gốc trên cùng câu (không có token giả): sai lệch tuyệt đối lớn nhất **0,00** trên ViT-B/32 (Mac) và ViT-L/14 (GPU Kaggle) — scripts/check_encoder_equivalence.py.
- Token “$” = 259 ở cả tokenizer đánh giá và tokenizer huấn luyện (“[$]” thêm vào tokenizer SDXL có mã 49408, được đổi thành 259).
- Checkpoint chính thức nạp khớp hoàn toàn (strict) và tái hiện FashionIQ lệch ≤ 0,77 điểm so với bài báo; chạy đánh giá hai lần cho thứ hạng từng truy vấn giống hệt.

- Các lỗi đã sửa trong mã gốc: gõ nhầm arg → args khi cắt gradient; truyền biến chưa định nghĩa khi đánh giá CIRCO; ép .half() gây lỗi trên CPU/MPS; ghi tham số list/None vào TensorBoard làm crash trên Kaggle.

Vận hành trên Kaggle

- Mỗi phiên tối đa 12 giờ, hạn mức 30 giờ GPU/tuần **tính theo giờ phiên** (đã kiểm chứng: 2:25 → 8:43 sau phiên 6,3 giờ dù dùng 2 GPU).
- Chạy 2 run song song, mỗi run 1 T4 với batch 512 (bộ nhớ đỉnh 11,4/15 GB). Không dùng 2 GPU cho 1 run vì khi đó InfoNCE chỉ thấy 256 mẫu âm mỗi GPU → khác cấu hình giữa các biến thể.
- Bộ lập lịch tự **ước lượng thời gian** run (học từ run đã xong) và không bắt đầu run mới nếu không kịp trước 11,3 giờ → không bao giờ bị Kaggle cắt giữa chừng và mất output. Mỗi phiên tự "mang theo" checkpoint, dữ liệu đánh giá của phiên trước (carry-forward).

Khắc phục nghẽn nạp dữ liệu

- Triệu chứng: chạy song song 4,5 giây/bước (dự kiến ~2,4) → 6,2 giờ/run. Đo trên caption thật: spaCy tốn 4,7 ms/câu, tokenizer 0,1 ms → GPU chờ CPU (Kaggle chỉ 4 vCPU cho 2 run).
- Giải pháp: spaCy xử lý theo lô (nlp.pipe) và bỏ thành phần NER, lemmatizer (không ảnh hưởng nhãn từ loại) → 1,51 ms/câu (nhanh ×3,1).
- Bằng chứng không đổi kết quả: từ khóa giống hệt trên 3.000 câu thật; DataLoader cùng seed cho các batch giống hệt từng token với mã cũ. Phương án bỏ cả parser bị loại vì làm khác kết quả ở 48/1.536 câu.
- Tốc độ sau tối ưu: **1,45 giây/bước khi 2 run song song** (2,0 giờ/run, trước đó 6,2 giờ) → 8 run trong một phiên thay vì 2.

Tệp liên quan trong dự án

- code/encode_with_pseudo_tokens.py, code/compat.py — encoder viết lại, tokenizer thay thế
- scripts/check_encoder_equivalence.py — kiểm chứng sai lệch 0
- code/loader.py (CaptionDataset.collate) — spaCy theo lô
- scripts/run_experiment.py, scripts/kaggle_jobs.py, notebooks/kaggle_lincir.ipynb — chạy thí nghiệm, lập lịch, carry-forward
- docs/decisions.md — lý do của từng quyết định

5. Kịch bản trình bày chi tiết

Slide	Ý chính cần nói
1	Giới thiệu đề tài, giảng viên, nhóm; tóm tắt khoảng 1 phút bằng ngôn ngữ thường (xem mục Tóm tắt dễ hiểu): ví dụ chiếc váy → bài toán; LinCIR học chỉ bằng văn bản, nhanh hơn 6–8 lần; nhóm dựng lại mã, B tốt hơn có ý nghĩa thống kê, A kém đi; nêu lộ trình trình bày.

2	Đọc bảng phân công: mỗi thành viên phụ trách một phần (Tâm: mã nguồn, Kaggle; Trung: cải tiến A, đường học; Tùng: tái hiện; Dũng: đánh giá, kiểm định, cải tiến B; Nou: lý thuyết, định tính, slides). Nói rõ khi hỏi đáp sẽ mời người phụ trách trả lời.
3	Bài toán CIR qua ví dụ chiếc váy: ảnh tham chiếu + câu "is shiny and silver with shorter sleeves" → ảnh đích; câu chỉ mô tả phần thay đổi. Bộ ba gán nhãn đắt và ít (46,6 nghìn, 28,8 nghìn) nên cần zero-shot. Bài báo LinCIR giải bài toán này chỉ bằng văn bản. Nêu câu hỏi của nhóm: sửa hàm mất mát (A) hay sửa nhiễu (B) có giúp LinCIR tốt hơn không.
4	Liên hệ chương I: CIR vẫn là truy xuất có xếp hạng. R@K = tỉ lệ truy vấn có ảnh đích trong K kết quả đầu (FashionIQ, 1 ảnh đúng). AP@K = Σ P@k·rel(k) / min(K, số ảnh đúng), mAP@K = trung bình trên truy vấn (CIRCO, nhiều ảnh đúng). Khác chương I: so khớp vector bằng cosine thay vì khớp từ. Giao thức: FashionIQ val, CIRCO val, kiểm định bootstrap và Wilcoxon theo từng truy vấn.
5	Ý tưởng chung của ZS-CIR dùng token giả: ảnh tham chiếu → bộ mã hóa ảnh → φ → token [$]; ghép vào "a photo of [$] that …" → bộ mã hóa văn bản → vector truy vấn → xếp hạng theo cosine. Chỉ học φ, CLIP giữ nguyên. Lần đầu nói: "ký hiệu đô-la trong ngoặc vuông là token giả".
6	Tóm tắt bài báo (Gu et al., NAVER, CVPR 2024). Vấn đề: Pic2Word, SEARLE cần ảnh khi huấn luyện, chỉ học câu mẫu cố định. Ý tưởng: Self-Masking Projection học φ chỉ từ văn bản, thêm nhiễu cho modality gap. Kết quả bài báo: tốt nhất 4 bộ đánh giá, nhanh hơn Pic2Word 6 lần, SEARLE 8,4 lần, vượt phương pháp có giám sát trên FashionIQ. Chuyển sang hai thành phần.
7	SMP: câu x_c → z_c; φ(z_c) → token ê_c; che tính từ và danh từ bằng [$], thay [$] bằng ê_c, mã hóa lại được ẑ_c; huấn luyện φ để ẑ_c gần z_c (MSE). Ví dụ "gray cat sleeps on a pillow" → "[$] sleeps on [$]": [$] học bản tóm tắt một từ của câu.
8	Modality gap: tâm ảnh và tâm văn bản cách nhau 0,82; ảnh và chú thích của chính nó chỉ có cosine 0,26. φ học trên văn bản nhưng suy luận nhận vector ảnh, nên LinCIR thêm nhiễu u·ε, u ~ U(0,1), ε ~ N(0,I), để φ chịu được độ lệch. Đây là thành phần B sẽ sửa.
9	Cải tiến A sửa hàm mất mát: MSE chỉ kéo câu đã che về câu gốc, không phạt khi nó cũng gần câu khác, trong khi đánh giá là xếp hạng. L = L_SMP + 0,5·InfoNCE, 511 mẫu âm trong batch, τ = 0,07; chi phí +2%/bước. Giả thuyết H1: A tăng R@K, mAP@K. Chưa nói kết quả.
10	Cải tiến B sửa nhiễu: u·ε đều theo mọi hướng, không dịch về phía vùng ảnh. n = 0,5·Δ + u·ε, Δ = tâm ảnh − tâm văn bản trên 5.000 cặp COCO val2017 (‖Δ‖ = 13,87), khác tập đánh giá nên không rò rỉ. H2: φ nhất quán hơn giữa ảnh và văn bản nên R@K, mAP@K tăng. H3: A+B sửa hai thành phần khác nhau nên có thể cộng hưởng.
11	Mã gốc viết cho transformers 4.34: viết lại encoder văn bản, thay tokenizer, sửa 4 lỗi; sai lệch 0,00 với CLIP gốc trên ViT-B/32 và ViT-L/14. Thực nghiệm: baseline, A, B, A+B × 3 seed, 5.000 bước, chỉ khác thành phần cải tiến; 2 GPU T4 trên Kaggle (12 giờ/phiên, 30 giờ/tuần); nạp dữ liệu nhanh gấp 3,1 lần, kết quả giống hệt.
12	Đánh giá checkpoint chính thức bằng mã đã chuyển đổi: lệch tối đa 0,77 điểm, R@10 trung bình 26,35 so với 26,28 của bài báo. Ba khác biệt: φ không có LayerNorm, thêm prompt Midjourney, từ khóa gồm cả mạo từ; nhóm theo mã vì mã tạo ra số liệu công bố.
13	So với baseline nhóm tự huấn luyện, không so với bài báo. B cao nhất ở mọi cột: R@10 +0,60 (p = 0,006), R@50 +1,27 (p < 0,001); B 5.000 bước vượt checkpoint

	chính thức 20.000 bước. A và A+B kém hơn trên CIRCO, khoảng tin cậy nằm dưới 0. Độ lệch chuẩn qua seed nhỏ (≤ 0,5).
14	Cosine giữa φ(ảnh) và φ(văn bản) của cùng cặp: B 0,84 > chính thức 0,81 > baseline 0,80 > A+B 0,69 > A 0,64, trùng thứ tự CIRCO mAP@5. InfoNCE buộc [$] mang đặc trưng phân biệt câu văn bản, không chuyển được sang ảnh. Đường học: A thấp hơn baseline ở mọi mốc.
15	Ví dụ FashionIQ: B xếp đúng ảnh đích hạng 1; nhiều ảnh trong top-5 đúng phần lớn yêu cầu nhưng không được gán nhãn (âm tính giả). Kết luận: H2 được ủng hộ, H1 và H3 bị bác bỏ. Đóng góp: dựng lại mã, tái hiện, hai cải tiến, kiểm định, phân tích cơ chế. Hạn chế: 5.000 bước, chưa quét λ, α, chưa có CIRR, CIRCO test; hướng C theo IDF.
16	Mời câu hỏi; nghe hết câu hỏi rồi trả lời hoặc mời người phụ trách: phương pháp, cải tiến A → Trung; tái hiện, bảng kết quả → Tùng; độ đo, kiểm định, cải tiến B, cơ chế → Dũng; bài toán, lý thuyết, ví dụ định tính → Nou; mã nguồn, Kaggle → Tâm.

6. Câu hỏi phản biện dự kiến

H1. Làm sao chắc encoder viết lại cho kết quả giống bản gốc?

→ So trực tiếp vector đầu ra với CLIPTextModelWithProjection gốc trên cùng đầu vào: sai lệch lớn nhất 0,00 trên cả ViT-B/32 và ViT-L/14. Thêm vào đó, checkpoint chính thức tái hiện khớp bài báo (lệch ≤ 0,77 điểm) và đánh giá lặp lại cho thứ hạng từng truy vấn giống hệt.

H2. Vì sao lấy vector tại EOS bằng argmax?

→ EOS có mã 49407, lớn nhất trong bảng từ, nên argmax của dãy mã chính là vị trí EOS. Mặt nạ nhân quả làm các token đệm sau EOS không ảnh hưởng.

H3. Tại sao chỉ 5.000 bước, có làm sai lệch kết luận không?

→ 20.000 bước trên T4 ≈ 13 giờ/run, vượt giới hạn phiên 12 giờ và hạn mức tuần. Mọi biến thể cùng 5.000 bước nên so sánh vẫn công bằng; số tuyệt đối có thể thấp hơn checkpoint 20.000 bước. Đường học (slide 14, Trung phân tích): R@10 chỉ tăng 0,25 điểm từ 4.000 đến 5.000 bước; baseline 5.000 bước kém checkpoint 20.000 bước 0,45 điểm R@10 và gần bằng trên CIRCO (10,47 so với 10,50).

H4. Chạy 2 run song song có ảnh hưởng kết quả?

→ Không: mỗi run một GPU riêng, cùng cấu hình; chỉ ảnh hưởng tốc độ. Kết quả phụ thuộc seed và dữ liệu, không phụ thuộc việc chạy cùng lúc.

H5. Bỏ NER và lemmatizer có đổi dữ liệu huấn luyện không?

→ Không. Hai thành phần này không ghi nhãn từ loại mà LinCIR dùng để chọn từ khóa. Đã kiểm tra từ khóa giống hệt trên 3.000 câu thật và batch giống hệt từng token với mã cũ. Riêng parser thì có ảnh hưởng nên được giữ lại.

H6. Tại sao không dùng 2 GPU cho 1 run cho nhanh?

→ Huấn luyện phân tán chia batch 512 thành 2 × 256; InfoNCE (cải tiến A) chỉ thấy mẫu âm trên GPU của mình → khác cấu hình với baseline. Chạy 2 run độc lập giữ batch 512 cho mọi biến thể.

H7. fp16 có làm khác kết quả so với bài báo?

→ Mã gốc cũng huấn luyện fp16. Khi đánh giá nhóm dùng fp32 như mã validate gốc; tái hiện khớp bài báo.

7. Tự kiểm tra trước buổi thuyết trình

- ☐ Vẽ lại được luồng: câu → token → nhúng → (thay [$]) → Transformer nhân quả → EOS → chiếu.
- ☐ Giải thích vì sao sai lệch 0 là bằng chứng đủ mạnh, và còn những bằng chứng nào khác.
- ☐ Nói được tốc độ trước/sau tối ưu và vì sao kết quả không đổi.
- ☐ Giải thích cách tính hạn mức Kaggle và cơ chế không để phiên bị cắt.

8. Đọc thêm

- Báo cáo: toàn bộ chương 1–6 (trình bày tất cả các phần); docs/decisions.md
- Bài báo LinCIR: Tóm tắt (Abstract), mục 1 (đóng góp), 3.1, 4.1
- CLIP (Radford 2021): mục 2.3 (kiến trúc encoder văn bản)
- README.md: bảng kết quả cần nhớ (nắm chắc trước khi trình bày slide 13–15)